```
err instanceof Error ? err.message : "An unknown error occurred";
      return { content:
[{ type: "text", text: `Error fetching joke: ${message}` }] };
    }
  }
);

async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
  console.log('Chuck Norris MCP Server running on stdio');
}
main()
```

Fetches a Chuck Norris fact from the API and handles errors gracefully

Creates a stdio transport and connects the MCP server so clients can call its tools

In this example, the MCP server is local to the Next.js application. The `StdioServerTransport` is a local interprocess communication mechanism: it runs the MCP server as a separate Node.js process and communicates over standard input/output (`stdio`). This means that the MCP server doesn't expose HTTP endpoints itself because it listens for requests via `stdio`, and the Next.js API route acts as the client that sends requests and receives structured responses.

The server executes the tool logic (e.g., fetching a joke from the Chuck Norris API) and returns the output in an MCP-compliant format. The AI model, here the Gemini model instantiated via the Vercel AI SDK, is completely isolated from direct API calls. The model only "knows" about the tools exposed by the MCP server; it cannot directly query external APIs. This separation ensures that all external API access is mediated by the MCP server, which simplifies security and modularity.

Figure 12.3 shows how the local MCP integration is structured within the Astra chat application. In this setup, the MCP server runs locally alongside the Next.js application. Communication between the API route and the MCP server occurs through standard input/output (`stdio`), a lightweight interprocess transport mechanism. This means the MCP server does not expose any HTTP endpoints, but instead, the Next.js API route launches and communicates with it directly, invoking tools and receiving structured responses.

The following listing shows the code on the Next.js side; we need to initiate both the server and the client inside the REST API route.

Listing 12.2 API route that serves the MCP server in Next.js

```
import { createGoogleGenerativeAI } from '@ai-sdk/google';
import { streamText, convertToModelMessages,
experimental_createMCPClient } from 'ai';
import { StdioClientTransport } from '@modelcontextprotocol/sdk/client/stdio';

export const maxDuration = 30;

const gemini = createGoogleGenerativeAI({
    apiKey: process.env.GEMINI_API_KEY || '',
});
```